Architecture Big Data appliquée à l'IA

Séance 5 : déployer un modèle de Machine Learning / IA

Franciscain Padonou

Séance 5 : déployer un modèle de Machine Learning / IA

Objectif : passer d'un modèle entraîné dans un notebook à un service et comprendre ce qui rend un système de ML fiable en production.

1. Le système ML au-delà du modèle

1.1 Le modèle est une petite partie du système

Un système de ML en production est fait surtout de code **autour** du modèle : collecte et validation des données, extraction des features, configuration, service, surveillance. Le code du modèle n'en est qu'une petite boîte au centre.

1.1 Le modèle est une petite partie du système (suite)

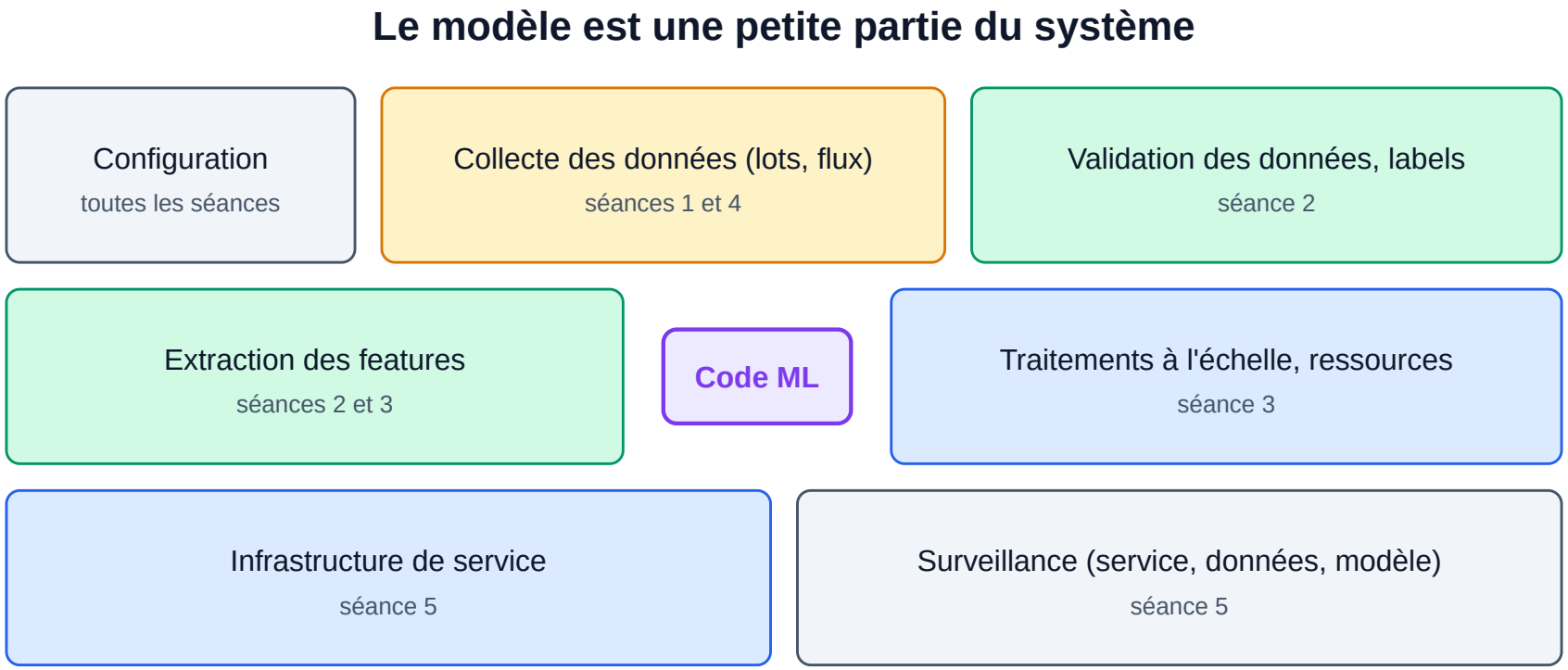


Schéma : le code du modèle (en violet) n'est qu'une petite partie du système ; chaque autre brique vient d'une séance du cours (d'après Sculley et al., 2015).

1.1 Le modèle est une petite partie du système (suite)

Pour ShopFlow, tout le cours y a contribué :

Brique	Séance
Données historiques, entrepôt	1
Labels (rétrofacturations), features historiques, versionnement des tables	2
Traitements à l'échelle	3
Flux temps réel	4
Modèle, service, surveillance	5

1.2 La dette technique propre au ML

- Modifier une feature, sa normalisation ou un seuil change le comportement de tout le modèle.
- **Dépendances aux données** plus fragiles que les dépendances au code : une table amont qui change casse le modèle sans erreur visible.
- **Boucles de rétroaction** : un modèle de fraude qui bloque des paiements modifie les données qui serviront à l'entraîner ensuite (les paiements bloqués n'auront jamais de rétrofacturation).

1.3 Règle n°1

En Machine Learning, ne pas hésiter à lancer un produit sans machine learning. Une règle simple (plus de 3 cartes sur une IP en 10 minutes est une fraude) est une base de comparaison indispensable, et souvent un premier système de production.

2. Ce que les données réelles imposent à un modèle

Dans un notebook, on évalue un modèle sur un jeu de données propre et figé. En production, presque tous les modèles de classification se heurtent aux trois mêmes difficultés, quel que soit le métier : la classe qui intéresse est rare, la bonne réponse arrive tard, et le monde change. ShopFlow et sa détection de fraude en sont un exemple parmi d'autres.

2.1 Choisir sa bonne métrique d'évaluation

Beaucoup de problèmes de ML cherchent à modéliser un événement rare : une maladie dans des examens médicaux, une panne dans des capteurs, un client qui va partir (*churn*), un défaut de remboursement de crédit, une fraude. On parle de **classes déséquilibrées**. Exemple ShopFlow : moins de 1 % des paiements sont frauduleux (environ 0,7 %). Un modèle qui répond toujours « non » a alors 99,3 % d'exactitude (*accuracy*) et ne sert à rien.

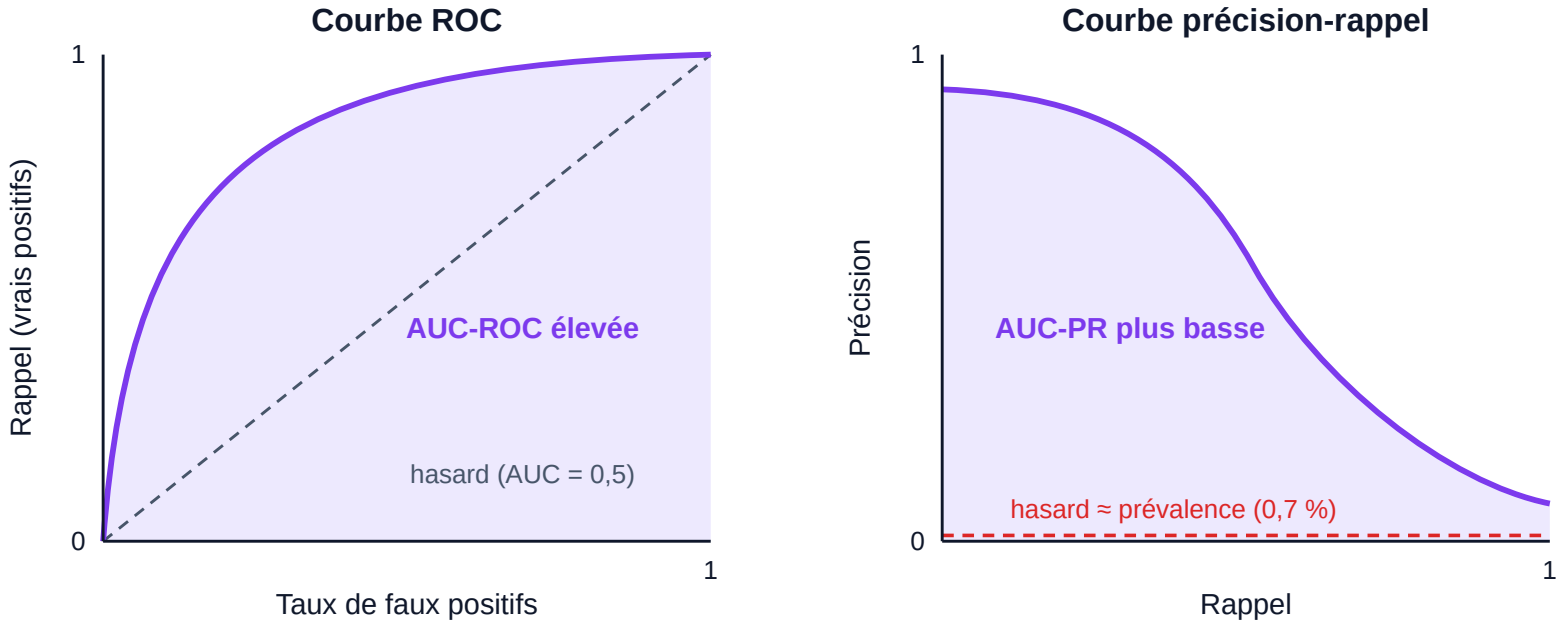
- Ne pas utiliser l'accuracy seule.
- **AUC-ROC** : mesure la capacité à bien classer les exemples, mais reste optimiste quand les négatifs sont très nombreux (les nombreux négatifs faciles la gonflent).
- **AUC-PR** (précision moyenne) : se concentre sur la classe rare ; c'est la métrique de référence quand cette classe est peu fréquente. Son niveau « hasard » vaut la proportion de positifs.

2.1 Choisir sa bonne métrique d'évaluation (suite)

- **Précision sur les k premiers** : quand des humains traitent les sorties du modèle (médecins, techniciens, analystes), ils n'en examinent qu'un nombre limité par jour ; ce qui compte est la part de vrais positifs parmi ces k cas.

2.1 Choisir sa bonne métrique d'évaluation (suite)

Un même modèle, deux courbes (allure illustrative)



ROC : optimiste quand les négatifs sont très nombreux. PR : centrée sur la classe rare (la fraude).

Schéma (illustratif, exemple ShopFlow) : pour le même modèle, la courbe ROC paraît excellente alors que la courbe précision-rappel, dont le hasard vaut la proportion de positifs (0,7 %), montre ce qu'il reste à gagner sur la classe rare.

2.2 La bonne réponse arrive tard

Pour beaucoup de modèles, le label (la vraie réponse) n'est connu que longtemps après la prédiction : un client part ou non dans les trois mois, un crédit est remboursé ou non après des années, une fraude n'est connue qu'à la **rétrofacturation**, 7 à 60 jours plus tard (exemple ShopFlow, séance 2). Conséquences générales :

- au moment d'entraîner, les données récentes n'ont pas encore de label fiable : il faut un **délai de maturation** entre la fin des données d'entraînement et la date de mise en production ;
- l'évaluation doit être **temporelle** : entraîner sur le passé, tester sur une période postérieure, comme le modèle sera utilisé. Un découpage aléatoire mélange le passé et l'avenir, et met les mêmes clients ou les mêmes événements des deux côtés : les résultats paraissent trop bons ;

2.2 La bonne réponse arrive tard (suite)

- quand le modèle (ou une règle) **agit** sur ce qu'il prédit, certains labels ne seront jamais observés : un paiement refusé ne sera jamais rétrofacturé, un crédit refusé ne sera jamais remboursé. Il faut en tenir compte dans le choix des données d'entraînement (pour ShopFlow : on entraîne sur les paiements autorisés).

2.2 La bonne réponse arrive tard (suite)

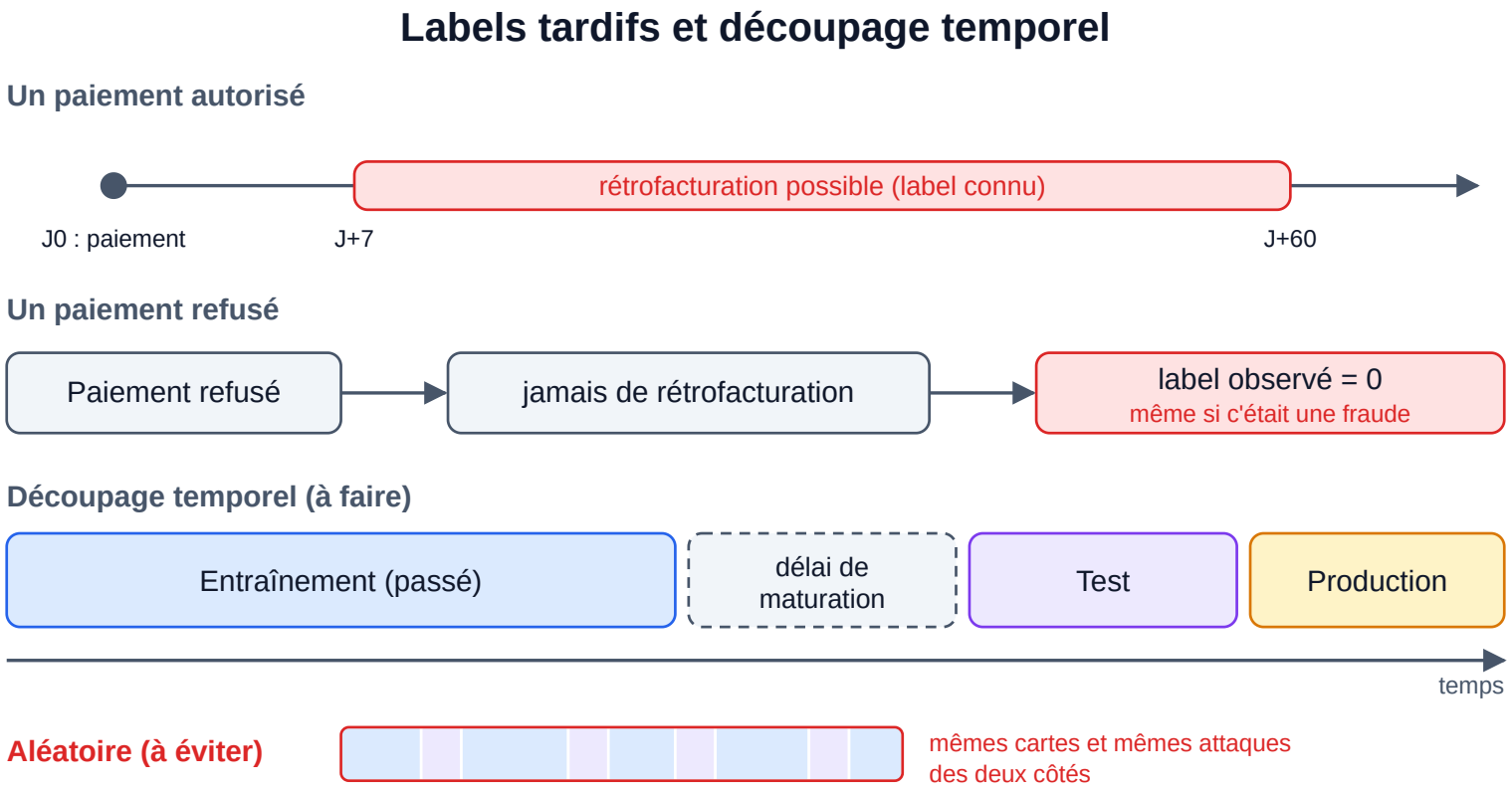


Schéma (exemple ShopFlow) : le label n'est connu qu'après un délai, d'où le délai de maturation entre entraînement et test ; un découpage aléatoire mélangerait les mêmes cas des deux côtés.

2.3 Drift

Un modèle apprend une photo du passé. Ensuite, les données d'entrée changent (nouveaux clients, saisonnalité, nouveau produit, capteur remplacé) et la relation entre les entrées et la bonne réponse peut changer aussi (nouvelles habitudes, concurrents, adversaires qui s'adaptent, comme les fraudeurs). On parle de **dérive des données** (*data drift*) et de **dérive du concept** (*concept drift*). Tout modèle en production doit donc être surveillé et ré-entraîné régulièrement (section 5).

3. Modes de déploiement

3.1 Trois modes

Mode	Principe	Latence	Exemple ShopFlow
Batch	Scorer périodiquement une table entière ; résultats stockés	Heures	Revue nocturne des paiements de la veille dans le lakehouse
En ligne (API)	Un service reçoit une requête et répond avec un score	Dizaines de ms	Le système de paiement appelle `/predict` avant d'autoriser
Embarqué dans le flux	Le modèle est chargé dans le consommateur Kafka	Millisecondes, sans réseau	Le scoreur lit `payments`, score et publie les alertes

Critères de choix : latence exigée, coût (un service en ligne tourne en permanence), couplage (une API se met à jour sans redéployer les consommateurs ; un modèle embarqué non), passage à l'échelle.

3.1 Trois modes (suite)

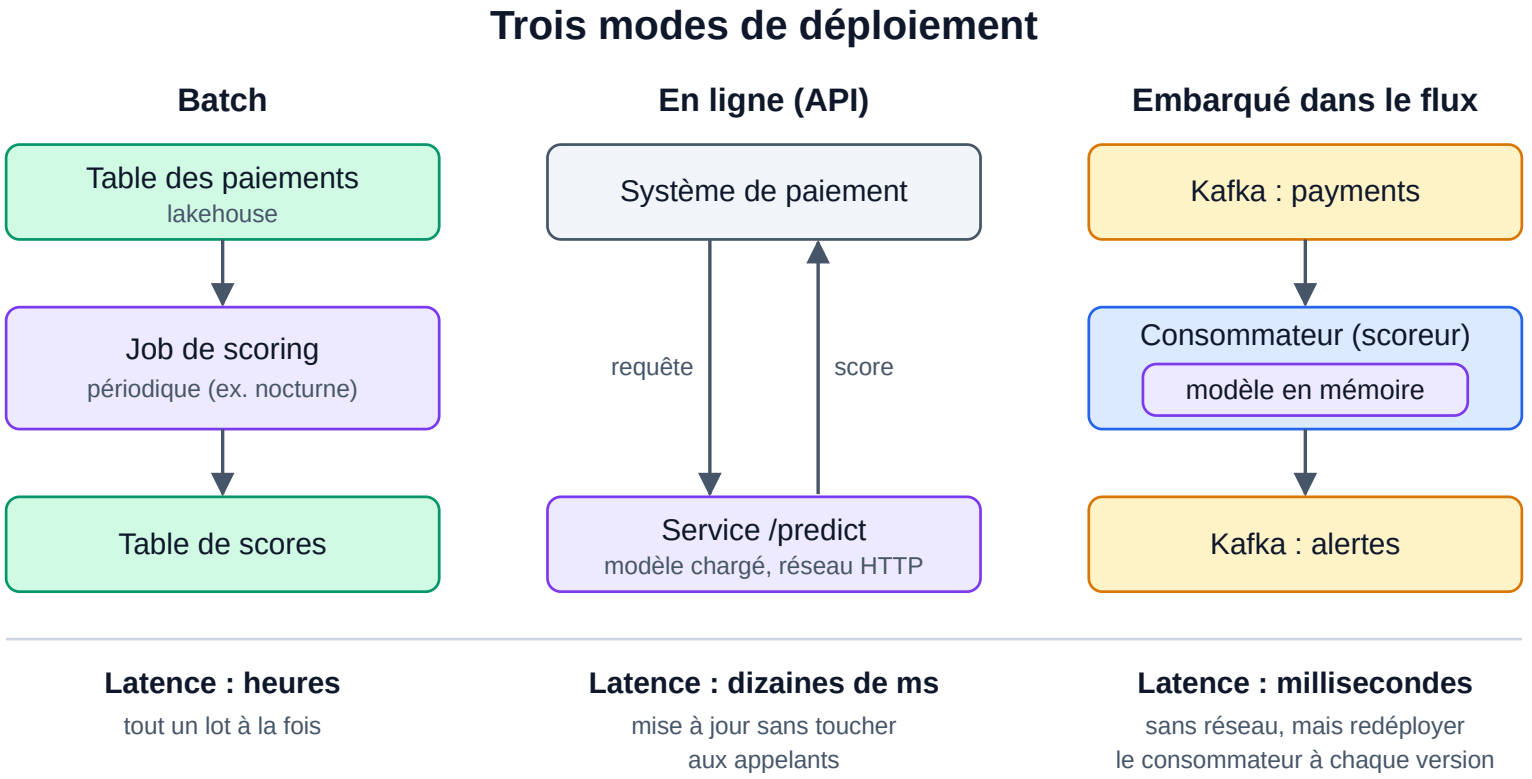


Schéma : où se trouve le modèle dans chaque mode, et la latence qui en résulte.

3.2 Le piège principal : l'écart entraînement / service

Le *training-serving skew* : les features calculées en production diffèrent de celles calculées pour l'entraînement. Causes typiques :

- deux implémentations différentes (SQL pour l'entraînement en séance 2, Python pour le flux en séance 5) qui divergent sur un détail (fenêtre incluse ou exclue, valeur manquante) ;
- un état incomplet au démarrage (le scoreur n'a pas encore 30 jours d'historique par carte) ;
- des versions de bibliothèques différentes (un modèle scikit-learn sérialisé avec `joblib` n'est garanti qu'avec la même version).

Remèdes : une seule définition des features (un **feature store** qui sert les mêmes valeurs hors ligne et en ligne), des tests qui comparent les deux calculs sur les mêmes données, l'enregistrement des features servies pour les réutiliser à l'entraînement suivant.

3.3 Feature stores

Un feature store garde les features sous deux formes : une table **hors ligne** (historique, pour l'entraînement, avec des jointures « à la date du paiement ») et un magasin **en ligne** à faible latence (dernière valeur par clé, pour le service). Exemples : Michelangelo chez Uber (2017), Databricks Feature Engineering, Vertex AI Feature Store, Feast (open source).

3.3 Feature stores (suite)

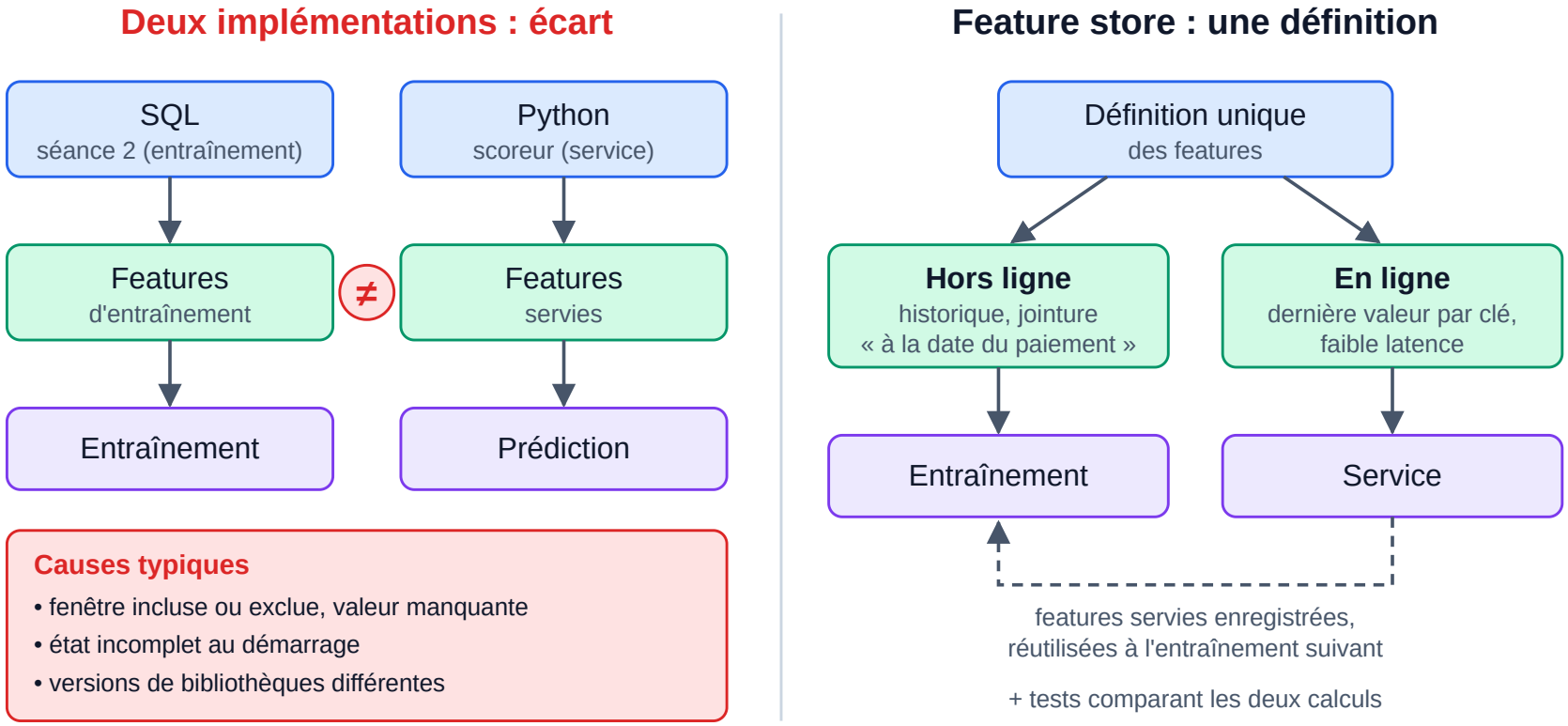


Schéma : deux implémentations des features divergent (écart entraînement / service) ; un feature store part d'une seule définition et la sert hors ligne et en ligne.

4. Cycle de vie et outillage

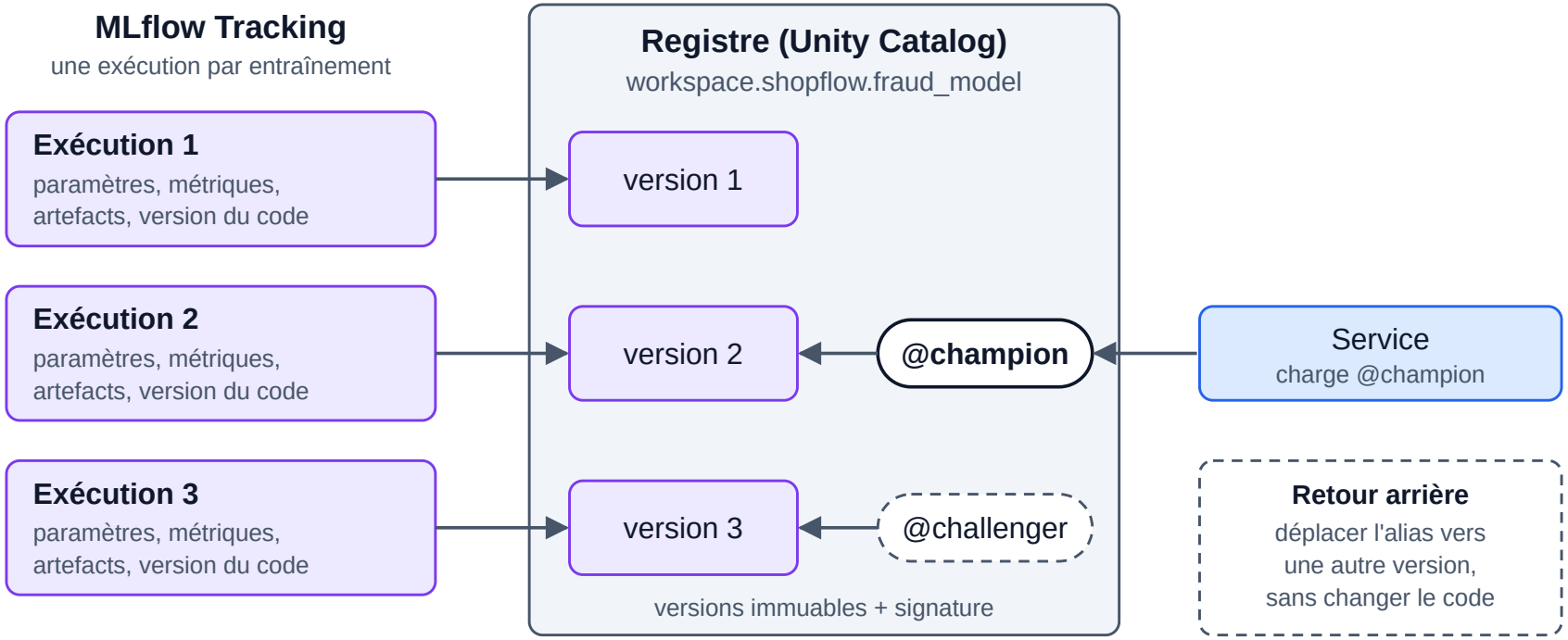
4.1 Suivre les expériences

Chaque entraînement est une **exécution** (*run*) qui enregistre ses paramètres, ses métriques, ses artefacts (le modèle, des graphiques) et la version du code et des données. **MLflow Tracking** est l'outil open source de référence ; il est intégré à Databricks.

4.2 Registre de modèles

Un registre donne à chaque modèle un nom, des **versions** immuables et des **alias** (`champion`, `challenger`) qui disent quelle version est en production. Sur Databricks, le registre est dans **Unity Catalog** : un modèle est un objet `catalogue.schéma.modèle`, avec les mêmes droits d'accès que les tables. Unity Catalog exige que chaque modèle ait une **signature** (schéma des entrées et des sorties).

4.2 Registre de modèles (suite)



Exemple : la version 2 est en production, la version 3 est en test.

Schéma : chaque exécution produit une version immuable ; le service lit l'alias `champion`, et revenir en arrière consiste à déplacer l'alias.

4.3 Servir

- **Conteneur** : l'image Docker fige le système, Python et les bibliothèques ; c'est la réponse à l'écart de versions.
- **Serverless** : **Cloud Run** exécute un conteneur HTTP, ajuste le nombre d'instances à la charge et descend à zéro sans trafic. Contrepartie : le **démarrage à froid** (première requête lente) ; on peut garder une instance minimale, qui est alors facturée en continu. L'offre gratuite de Cloud Run couvre 2 millions de requêtes par mois, largement assez pour le TP.
- **Services managés de ML** (Vertex AI, Databricks Model Serving, SageMaker) : déploiement depuis le registre, versions, répartition du trafic ; plus simples mais plus coûteux.

4.3 Servir (suite)

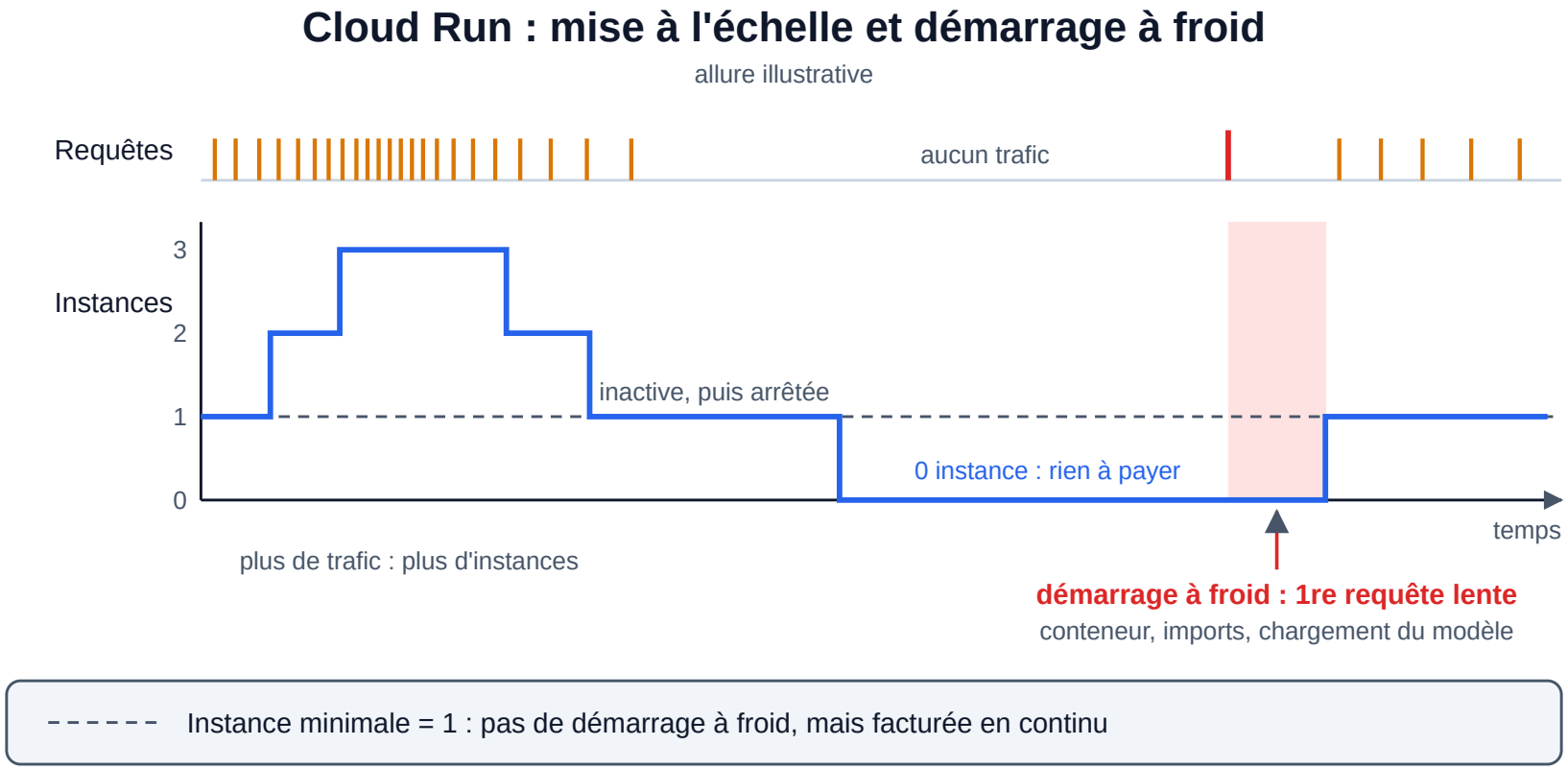


Schéma (illustratif) : sans trafic, Cloud Run descend à zéro instance ; la requête suivante subit un démarrage à froid, sauf si l'on garde une instance minimale facturée en continu.

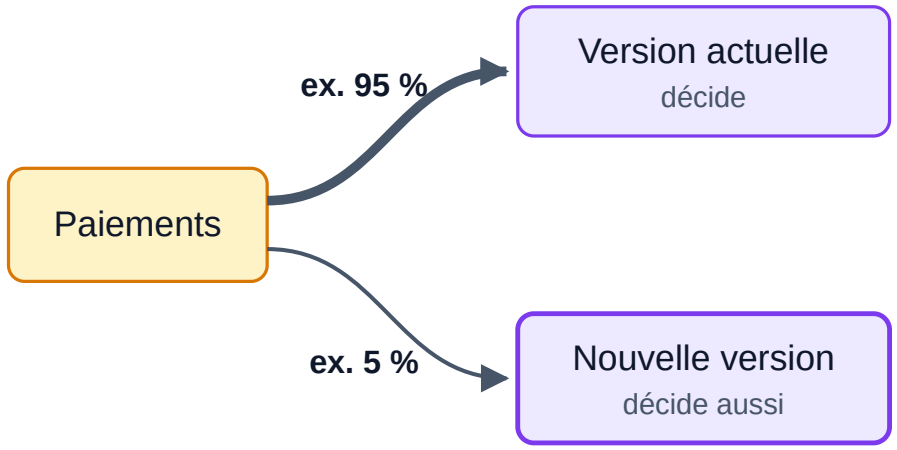
4.4 Déployer prudemment

Comme pour du logiciel : déploiement **canari** (une petite part du trafic vers la nouvelle version), **shadow** (la nouvelle version score sans décider, on compare), retour arrière rapide grâce aux alias du registre.

4.4 Déployer prudemment (suite)

Canari

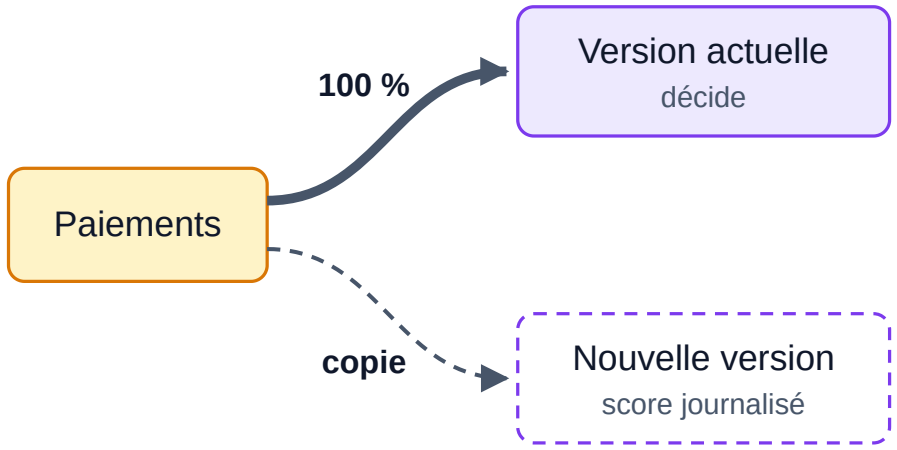
une petite part du trafic vers la nouvelle version



On compare les résultats des deux versions.

Shadow

la nouvelle version score sans décider



On compare les scores, sans risque client.

Retour arrière rapide : déplacer l'alias du registre.

Schéma : en canari, la nouvelle version décide pour une petite part du trafic ; en shadow, elle voit tout le trafic mais ne décide jamais.

5. Surveiller en production

5.1 Trois niveaux

Niveau	Exemples d'indicateurs	Disponible
Service	Latence p50/p99, taux d'erreurs, nombre d'instances	Immédiatement (Cloud Run le fournit)
Données et prédictions	Distribution des features et des scores, part de valeurs manquantes, volume d'alertes	Immédiatement
Performance du modèle	Précision des alertes, AUC-PR	Quand les labels arrivent (semaines)

Comme les labels arrivent tard, on surveille d'abord les **entrées** et les **sorties** du modèle.

5.2 Les dérives

Un modèle a appris sur les données d'une période donnée. Une **dérive** est un changement, après l'entraînement, des données que le modèle voit en production. On en distingue trois sortes :

- **Dérive des données** (*data drift*, ou *covariate shift*) : la distribution des entrées change. Exemple : les montants des paiements augmentent pendant le Black Friday, de nouveaux clients arrivent d'un autre pays, un capteur est remplacé.
- **Dérive des labels** (*label shift*) : la proportion de la classe à prédire change. Exemple : la part de fraudes double pendant une vague d'attaques.
- **Dérive du concept** (*concept drift*) : le lien entre les entrées et la bonne réponse change. Exemple : les fraudeurs changent de méthode, et un paiement qui avait l'air normal hier est frauduleux aujourd'hui.

5.2 Les dérives (suite)

Pour **détecter** une dérive des données sans attendre les labels, on compare la distribution d'une variable (une feature ou le score du modèle) en production à sa distribution à l'entraînement, et on résume l'écart par un indicateur : plus il est grand, plus la production s'éloigne de ce que le modèle connaît. Les indicateurs les plus courants s'appellent **PSI** (*Population Stability Index*, très utilisé en crédit et en fraude), test de **Kolmogorov-Smirnov** et distance de **Wasserstein** ; des bibliothèques comme Evidently les calculent. Une dérive détectée n'impose pas de ré-entraîner : on en cherche d'abord la cause (bug dans les données amont, changement de catalogue, vraie évolution des clients).

5.3 Prêt pour la production ?

Des ingénieurs de Google proposent une grille de 28 tests : tests des features et des données, du développement du modèle, de l'infrastructure, et de la surveillance. Elle sert de liste de contrôle pour le dossier d'architecture.

- Breck et al., *The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction*, IEEE Big Data 2017.
- Evidently : méthodes de détection de dérive ; Cloud Run : métriques de surveillance.

6. Ouverture : l'IA générative dans ce système

Un modèle de langage n'a pas sa place dans la **décision** (latence, coût, reproductibilité, explicabilité réglementaire). Il peut aider les **analystes** : résumer en langage naturel pourquoi un paiement a été signalé, à partir des features et de l'historique de la carte, ou aider à écrire des requêtes d'investigation. La décision reste au modèle tabulaire et aux règles, testés et surveillés.

6.1 Un LLM en production se surveille aussi

Un modèle de langage (LLM) déployé en production n'échappe à rien de ce qui précède : il fait partie d'un système (section 1), il faut mesurer sa performance avant de le déployer, puis le surveiller en continu (section 5). Ce qui change, c'est **quoi** mesurer : une réponse en texte libre n'a pas une seule « bonne réponse » qu'on compare à un label.

6.1 Un LLM en production se surveille aussi (suite)

Avant le déploiement : évaluer.

- Construire un **jeu d'évaluation** : quelques centaines de questions représentatives (par exemple « pourquoi ce paiement a-t-il été signalé ? »), avec la réponse attendue ou les éléments qu'elle doit contenir.
- Mesurer plusieurs critères : la **fidélité** (la réponse ne dit que ce qui est dans les données fournies, sans rien inventer : ce qu'on appelle une hallucination), la **pertinence**, le respect du **format** attendu, le **ton**.
- Noter automatiquement quand c'est possible (règles simples, comparaison à la réponse attendue, ou un second LLM utilisé comme « juge »), et faire relire un échantillon par des humains, qui restent la référence. Un LLM juge a ses propres biais : il faut vérifier qu'il est d'accord avec les humains.

6.1 Un LLM en production se surveille aussi (suite)

- Rejouer cette évaluation à **chaque changement** : nouveau modèle, nouvelle consigne (*prompt*), nouvelles données. Le prompt se versionne comme du code ou comme un modèle dans un registre.

En production : surveiller. On retrouve les trois niveaux de la section 5.1, avec des indicateurs propres aux LLM :

Niveau	Exemples d'indicateurs pour un LLM
Service	Latence (temps avant le premier mot, temps total), taux d'erreurs, disponibilité de l'API du fournisseur
Coût	Nombre de tokens (morceaux de mots) en entrée et en sortie par requête, coût par jour ; il varie avec la longueur des questions et des réponses
Données et réponses	Sujets des questions (apparaît-il des demandes nouvelles ?), longueur des réponses, part de refus, réponses vides ou mal formées
Qualité	Notes du juge sur un échantillon de réponses réelles, relecture humaine, retours des utilisateurs (pouce levé ou baissé)
Sécurité	Tentatives d'**injection de prompt** (un texte qui détourne les consignes), fuite de données personnelles, contenus inappropriés

6.1 Un LLM en production se surveille aussi (suite)

Pour pouvoir analyser un problème, chaque appel est **tracé** : la question, le contexte fourni, la réponse, la version du modèle et du prompt, la durée et le coût. C'est l'équivalent de la table `shopflow_rt.scores` du TP 5.

Un piège propre aux LLM : le modèle change sans prévenir. Quand on appelle le modèle d'un fournisseur par API, celui-ci peut le mettre à jour. Une étude a montré que les réponses d'un même service pouvaient changer nettement en quelques mois. D'où trois réflexes : fixer la version exacte du modèle quand le fournisseur le permet, rejouer l'évaluation à chaque nouvelle version, et surveiller la qualité en continu.

Sources : 6.1 Un LLM en production se surveille aussi

Sources : Zinkevich, *Rules of ML*, règles 1 à 3 (commencer simple, mesurer). ; Huyen, *AI Engineering: Building Applications with Foundation Models*, O'Reilly, 2025, chap. 3 et 4 (évaluation) et chap. 10 (architecture, surveillance et retours utilisateurs). ; Zheng et al., *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*, NeurIPS 2023. ; Chen, Zaharia, Zou, *How Is ChatGPT's Behavior Changing over Time?*, 2023. ; MLflow pour l'IA générative (traçage, évaluation, registre de prompts). ; OWASP, *Top 10 for Large Language Model Applications*.

L'architecture ShopFlow complète

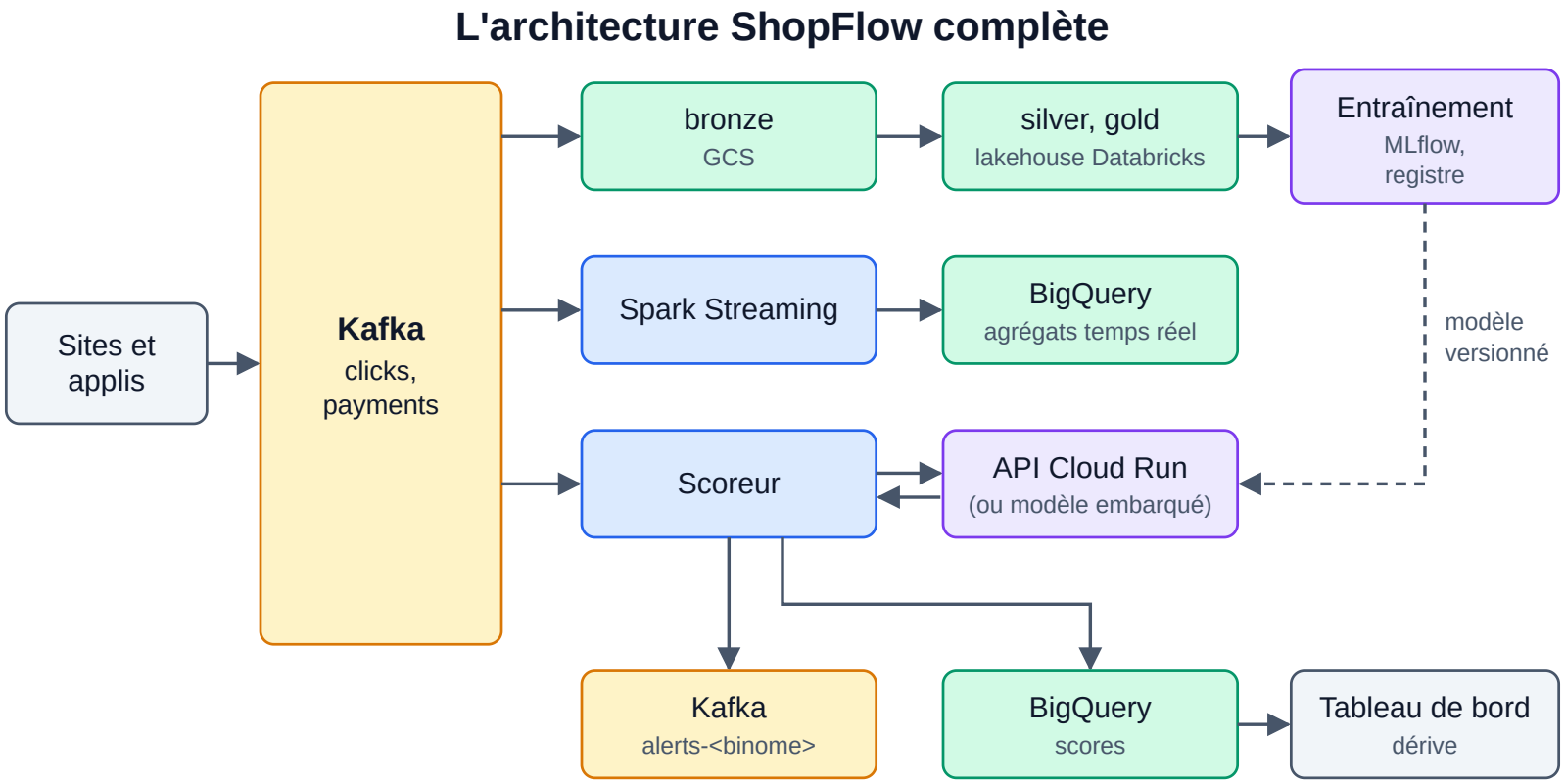


Schéma : les trois branches issues de Kafka (lot vers le lakehouse et l'entraînement, agrégats en flux, scoring), et le modèle versionné qui rejoint le service (flèche pointillée).